

NutriBalance

Documentação Técnica Completa

Como o sistema funciona, e por que foi feito assim

Trabalho de Conclusão de Curso

Ensino Médio Integrado — Desenvolvimento de Sistemas

Ryan da Silva Augusto

Julho de 2026

Sumário

1. Visão geral do sistema

1. O que é o NutriBalance
2. As três camadas e por que separá-las
3. Por que o site não funciona com clique duplo
4. O ciclo completo de uma requisição
5. Por que o PHP não é misturado com o HTML

2. O banco de dados

1. Por que um banco e não arquivos
2. A tabela alimentos
3. A tabela usuarios
4. A tabela medicos
5. A tabela redefinicoes_senha
6. Conceitos de banco usados no projeto
7. Por que o banco cabe em um arquivo só

3. O back-end em PHP

1. O bootstrap comum
2. Conexão com PDO
3. Prepared statements e SQL Injection
4. Sessões: como o servidor sabe quem está logado
5. Senhas: hash, nunca texto puro
6. Códigos de status HTTP
7. Percurso de cada endpoint
8. Por que o servidor recalcula tudo
9. Upload de imagem com validação real
10. Recuperação de senha: token de uso único

4. A matemática nutricional

1. Taxa metabólica basal (Mifflin-St Jeor)
2. Fator de atividade e TDEE
3. Déficit e superávit calórico
4. Macronutrientes e os fatores 4/4/9
5. As divisões 50/20/30 e 40/35/25
6. IMC e faixa de peso ideal
7. Previsão de prazo até o objetivo
8. Exemplo numérico completo

5. O front-end

- 6. Dados no navegador e versionamento**
- 7. Convenção de código em português**
- 8. Mapa de arquivos e como rodar**
- 9. Perguntas prováveis da banca**
- 10. Limitações honestas e próximos passos**
- 11. Glossário**

1. Visão geral do sistema

1.1 O que é o NutriBalance

O NutriBalance é um site de acompanhamento nutricional. O usuário cria uma conta, informa seus dados corporais e seu objetivo, e o sistema calcula quantas calorias e quantos gramas de cada macronutriente ele deve consumir por dia. A partir daí, ele registra as refeições do dia e acompanha quanto falta para bater a meta, além de registrar peso e cintura ao longo do tempo para ver a evolução em gráfico.

A referência é o MyFitnessPal, sem a funcionalidade de leitura de código de barras. A busca de alimentos usa a **TACO — Tabela Brasileira de Composição de Alimentos, 4ª edição**, publicada pela UNICAMP, com 597 alimentos importados para um banco MySQL local.

POR QUE ISSO IMPORTA SABER

Usar a TACO em vez de uma API estrangeira é uma decisão de projeto defensável: os dados são de alimentos brasileiros, medidos por instituição pública nacional, e ficam no banco local — o site não depende de internet nem de um serviço de terceiros que pode sair do ar no dia da apresentação.

1.2 As três camadas e por que separá-las

O sistema é dividido em três camadas. Entender essa divisão é o que permite explicar qualquer parte do projeto.

Camada	Onde roda	Tecnologia	Responsabilidade
Front-end	No navegador do usuário	HTML, CSS, JavaScript	Mostrar a interface, capturar cliques e digitação, desenhar gráficos
Back-end	No servidor (Apache)	PHP	Validar dados, autenticar, calcular metas, conversar com o banco
Banco de dados	No servidor (MySQL)	MySQL / MariaDB	Guardar de forma permanente: contas, perfis, medições, alimentos

A razão de separar é **confiança**. Tudo o que roda no navegador está sob controle do usuário: ele pode abrir o DevTools, alterar o JavaScript, mudar valores antes de enviar. Por isso, nada que venha do navegador pode ser tratado como verdade. O back-end existe para ser a autoridade — ele revalida tudo e é o único que pode escrever no banco.

REGRA FUNDAMENTAL

Nunca confie no cliente. No NutriBalance isso aparece de forma concreta: o JavaScript calcula a meta de calorias para mostrar a prévia ao vivo na tela, mas quando o usuário salva, o PHP *recalcula tudo do zero* e ignora o valor que o navegador mandou. Se alguém editar o JavaScript para dizer "minha meta é 9000 kcal", o servidor simplesmente não usa esse número.

1.3 Por que o site não funciona com clique duplo

Um site feito só de HTML, CSS e JavaScript pode ser aberto com clique duplo no arquivo — o navegador lê o arquivo do disco e pronto. O NutriBalance não, e a razão é o PHP.

PHP é uma linguagem que roda **no servidor**, não no navegador. O navegador não sabe o que fazer com um arquivo `.php`; ele espera receber HTML ou JSON já pronto. Quem lê o arquivo `.php`, executa o código, conversa com o banco e devolve o resultado é o **Apache** (o servidor web) junto com o interpretador PHP.

Por isso o projeto exige o XAMPP, que empacota Apache, MySQL e PHP juntos, e o site precisa ser acessado por `http://localhost/nutribalance/` — nunca por `file:///C:/.../index.html`.

1.4 O ciclo completo de uma requisição

Este é o caminho que os dados percorrem quando o usuário busca um alimento. Se você entender este fluxo, entende o sistema inteiro, porque todos os outros seguem o mesmo padrão.

1. O usuário digita "arroz" no campo de busca.
2. NAVEGADOR (`js/alimentos.js`)
O evento "input" dispara a função `executarBusca()`.
Ela espera 400 ms sem novas teclas (debounce) e então chama:
`fetch("api/buscar.php?q=arroz")`
3. REDE
Uma requisição HTTP GET viaja até o Apache.
4. SERVIDOR (`api/buscar.php`)
O Apache reconhece a extensão `.php` e entrega ao interpretador.
O PHP lê o parâmetro `q`, monta a consulta SQL com placeholders (`?`),
e pede ao MySQL.
5. BANCO (MySQL)
Executa: `SELECT descricao, categoria, energia_kcal, proteina, carboidrato, lipideos FROM alimentos WHERE energia_kcal IS NOT NULL AND descricao LIKE ?`
Devolve as linhas encontradas.
6. SERVIDOR (`api/buscar.php`)
Converte cada linha para o formato que o site usa e responde:
`[{ "nome": "...", "kcal100": 128.26, "proteina100": 2.521, ... }]`
7. REDE
O JSON volta pelo mesmo caminho.
8. NAVEGADOR (`js/alimentos.js`)
`mostrarResultados()` transforma o JSON em botões HTML
e injeta na tela com `innerHTML`.
9. O usuário vê a lista de alimentos.

VOCABULÁRIO

Requisição é o pedido que o navegador faz ao servidor. **Resposta** é o que volta. **Endpoint** é cada endereço do servidor que aceita requisições — no projeto, cada arquivo dentro de `api/` é um endpoint. **JSON** é o formato de texto usado para trocar dados estruturados entre os dois lados; é o "idioma comum" entre o PHP e o JavaScript.

1.5 Por que o PHP não é misturado com o HTML

Existem duas maneiras consagradas de construir um site com PHP, e o NutriBalance usa a segunda. Vale explicar as duas, porque a diferença aparece em todo arquivo do projeto.

Renderização no servidor. Um único arquivo `.php` faz tudo: recebe o formulário, consulta o banco e imprime o HTML da resposta. A cada ação, a página inteira recarrega. É o modelo do WordPress e do Laravel Blade, e continua sendo usado em produção no mundo inteiro.

```
<?php
if (isset($_POST['ok'])) {           // 1. recebe o formulario
    $sql = "SELECT ... WHERE ...";    // 2. consulta o banco
    $erros[] = "E-mail invalido";     // 3. acumula mensagens
}
?>
<form method="POST">                <!-- 4. e desenha a tela -->
    <input name="email">
</form>
```

API com front-end separado. O PHP não imprime tela nenhuma: ele responde apenas dados, em JSON. Quem monta a interface é o JavaScript, no navegador. A página não recarrega.

```
// api/login.php – nao existe uma linha de HTML aqui
responder(['ok' => true, 'perfil_completo' => $perfilCompleto]);

// js/auth.js – o navegador pede, recebe os dados e decide o que fazer
const { status, dados } = await enviarApi("api/login.php", { email, senha });
if (status === 200 && dados.ok) irAposAutenticar(dados.perfil_completo);
```

NENHUMA DAS DUAS ESTÁ ERRADA

São arquiteturas diferentes para problemas diferentes, não uma certa e uma errada. A primeira é mais simples de escrever e ótima para páginas de conteúdo. A segunda separa responsabilidades e é o que se usa quando a interface precisa reagir sem recarregar.

A escolha aqui foi ditada pela natureza do NutriBalance: o dashboard é uma tela viva. O usuário adiciona um alimento e as barras de macros se movem; ele troca de dia e a lista muda; ele registra uma medição e o gráfico se redesenha. Se cada uma dessas ações recarregasse a página inteira, o app seria lento e perderia a posição da rolagem a cada clique.

Há um segundo motivo, mais prático: o mesmo endpoint serve a mais de um consumidor. O `sessao.php` é chamado pelo dashboard, pela tela de personalização e pela guarda de todas as páginas. Como ele devolve

dados e não HTML, cada tela usa o que precisa. Se ele imprimisse a tela do dashboard, não serviria para as outras.

Consistência vale mais que a escolha em si

O critério que realmente importa é o projeto inteiro seguir o mesmo padrão. Um sistema com dez endpoints em JSON e um décimo primeiro que imprime HTML é mais difícil de entender que um sistema inteiro em qualquer um dos dois modelos — porque quem lê precisa descobrir, arquivo por arquivo, qual regra vale ali.

Por isso os onze endpoints de `api/` respondem todos em JSON, e dez deles seguem exatamente o mesmo esqueleto: incluem o `_bootstrap.php`, exigem o método HTTP, validam a entrada, falam com o banco por PDO e respondem com `responder()`. Aprendido um, os outros nove se leem sozinhos.

A exceção é o `buscar.php`, que consulta a tabela de alimentos e não precisa de sessão: ele foi escrito antes do `_bootstrap.php` existir e define os próprios cabeçalhos. Devolve JSON como os demais, mas não compartilha os auxiliares — é uma inconsistência pequena e conhecida, registrada aqui em vez de escondida.

O QUE DE FATO É ERRO, EM QUALQUER ARQUITETURA

A arquitetura é escolha. Três coisas não são, e valem para os dois modelos igualmente: concatenar valor do usuário dentro do SQL (abre SQL Injection), guardar senha com `md5()` (quebrado há anos, reversível em segundos por sites públicos) e confiar em validação feita apenas no navegador. O projeto usa prepared statements, `password_hash()` e revalidação no servidor — as seções 3.3, 3.5 e 3.8 detalham cada um.

2. O banco de dados

2.1 Por que um banco e não arquivos

Seria possível guardar os dados em arquivos de texto. Não se faz isso por quatro motivos práticos:

- **Busca eficiente.** Encontrar "arroz" entre 597 alimentos num arquivo exige ler o arquivo inteiro. O banco usa índices e vai direto.
- **Concorrência.** Se duas pessoas salvarem ao mesmo tempo, um arquivo pode ser corrompido. O banco controla acesso simultâneo.
- **Integridade.** O banco garante regras: um e-mail não pode repetir, uma medição precisa pertencer a um usuário que existe.
- **Consultas combinadas.** "Todas as medições do usuário 9 ordenadas por data" é uma linha de SQL.

2.2 A tabela alimentos

Guarda a TACO. Uma linha por alimento, e **todos os valores nutricionais são por 100 g de parte comestível** — essa é a unidade padrão da tabela, e é por isso que o código trabalha o tempo todo com campos chamados `kcal100`, `proteina100`, `carbo100` e `gordura100`.

```
CREATE TABLE alimentos (  
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  numero_taco INT UNSIGNED NULL,      -- numero do alimento na TACO  
  categoria   VARCHAR(80) NULL,      -- grupo (ex: "Cereais e derivados")  
  descricao   VARCHAR(255) NOT NULL,  -- nome do alimento  
  
  energia_kcal DECIMAL(8,2) NULL,    -- kcal por 100 g  
  proteina     DECIMAL(8,3) NULL,    -- g  
  lipideos     DECIMAL(8,3) NULL,    -- g (gordura)  
  carboidrato  DECIMAL(8,3) NULL,    -- g  
  ... (minerais e vitaminas)  
  
  PRIMARY KEY (id),  
  KEY idx_descricao (descricao),  
  FULLTEXT KEY ft_descricao (descricao)  
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;
```

DECISÃO EXPLICADA: COMO OS VALORES ESPECIAIS DA TACO FORAM TRATADOS

A planilha original da TACO usa códigos em vez de números em alguns casos. Foram convertidos assim na importação:

- **NA** (não disponível), ***** (não aplicável) e células em branco viraram **NULL** — que em banco significa "não existe informação", diferente de zero.
- **Tr** (traço, quantidade insignificante mas medida) virou **0**, porque nesse caso a informação existe e o valor é praticamente nulo.

A distinção entre `NULL` e `0` não é detalhe: a consulta de busca filtra `WHERE energia_kcal IS NOT NULL` justamente para não mostrar alimentos sem informação calórica, que quebrariam o cálculo da refeição.

2.3 A tabela `usuarios`

Guarda a conta e o perfil. Note que ela mistura duas coisas: os dados de login e os dados de personalização.

```
CREATE TABLE IF NOT EXISTS usuarios (  
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  nome        VARCHAR(120) NOT NULL,  
  email       VARCHAR(190) NOT NULL,  
  senha_hash  VARCHAR(255) NOT NULL,      -- nunca a senha, so o hash  
  foto        VARCHAR(255) NULL,          -- caminho do arquivo, nao a imagem  
  
  -- personalizacao (NULL ate o usuario completar o onboarding)  
  sexo        ENUM('M','F') NULL,  
  idade       INT UNSIGNED NULL,  
  altura_cm   INT UNSIGNED NULL,  
  peso_kg     DECIMAL(5,1) NULL,  
  cintura_cm  DECIMAL(5,1) NULL,  
  peso_alvo   DECIMAL(5,1) NULL,  
  atividade   ENUM('sedentario','leve','moderado','intenso','muito_intenso') NULL,  
  meta        ENUM('perder','manter','ganhar') NULL,  
  objetivo    ENUM('manter','definir') NULL,  
  
  -- metas calculadas (meta_kcal IS NOT NULL == perfil completo)  
  meta_kcal   INT UNSIGNED NULL,  
  meta_carbo  INT UNSIGNED NULL,  
  meta_proteina INT UNSIGNED NULL,  
  meta_gordura INT UNSIGNED NULL,  
  
  criado_em   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_email (email)  
);
```

Três decisões que valem explicar

1. As colunas de personalização aceitam `NULL`. Quando o usuário se cadastra, ele só informa nome, e-mail e senha. O perfil vem depois, na tela de personalização. Enquanto não vier, essas colunas ficam vazias. O sistema usa isso como sinal: `meta_kcal IS NOT NULL` significa "perfil completo". É por isso que, ao entrar, quem ainda não preencheu é redirecionado para a personalização em vez do dashboard.

2. A foto guarda o caminho, não a imagem. A coluna `foto` guarda um texto como `uploads/fotos/foto_9_73119afeb9ac.png`. O arquivo em si fica no disco. Guardar imagens dentro do banco é possível, mas deixa o banco pesado, lento para copiar e difícil de servir — o navegador precisaria de um endpoint PHP só para devolver cada imagem, em vez de o Apache entregar o arquivo diretamente.

3. As metas calculadas ficam salvas, não são recalculadas a cada acesso. Poderiam ser derivadas do peso e da altura toda vez. Salvá-las tem duas vantagens: o dashboard não precisa refazer a

conta a cada carregamento, e fica registrado qual era a meta vigente — inclusive se o usuário ajustou os macros manualmente.

2.4 A tabela `medicoes`

```
CREATE TABLE IF NOT EXISTS medicoes (  
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  usuario_id  INT UNSIGNED NOT NULL,  
  data        DATE          NOT NULL,  
  peso_kg     DECIMAL(5,1)  NOT NULL,  
  cintura_cm  DECIMAL(5,1)  NULL,          -- opcional  
  criado_em   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_usuario_data (usuario_id, data),  
  CONSTRAINT fk_medicoes_usuario  
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE  
);
```

A DISTINÇÃO MAIS IMPORTANTE DO PROJETO

`usuarios.peso_kg` é o **peso atual**. A tabela `medicoes` é o **histórico**. São coisas diferentes com propósitos diferentes: o peso atual alimenta o cálculo da meta; o histórico alimenta o gráfico de evolução. Quando o usuário registra uma medição nova, o sistema atualiza os dois — grava no histórico e sincroniza o peso atual com a medição mais recente por data.

A chave `UNIQUE (usuario_id, data)` garante **uma medição por dia por usuário**. Isso permite usar a técnica de *upsert*: se já existe registro naquele dia, atualiza; se não existe, insere. Em SQL:

```
INSERT INTO medicoes (usuario_id, data, peso_kg, cintura_cm)  
VALUES (?, ?, ?, ?)  
ON DUPLICATE KEY UPDATE  
  peso_kg = VALUES(peso_kg),  
  cintura_cm = VALUES(cintura_cm)
```

Sem a chave única, registrar duas vezes no mesmo dia criaria dois pontos no gráfico na mesma data — visualmente confuso e sem sentido.

2.5 A tabela `redefinicoes_senha`

```
CREATE TABLE IF NOT EXISTS redefinicoes_senha (  
  id          INT UNSIGNED NOT NULL AUTO_INCREMENT,  
  usuario_id  INT UNSIGNED NOT NULL,  
  token_hash  CHAR(64)      NOT NULL,    -- sha256 do token enviado no link  
  expira_em   DATETIME      NOT NULL,    -- prazo de validade  
  usado_em    DATETIME      NULL,        -- NULL = ainda nao foi usado  
  criado_em   TIMESTAMP     NOT NULL DEFAULT CURRENT_TIMESTAMP,  
  
  PRIMARY KEY (id),  
  UNIQUE KEY uq_token_hash (token_hash),  
  KEY idx_usuario_pendente (usuario_id, usado_em),  
  CONSTRAINT fk_redefinicoes_usuario  
    FOREIGN KEY (usuario_id) REFERENCES usuarios(id) ON DELETE CASCADE  
);
```

Guarda os pedidos de "esqueci minha senha". Cada linha representa um link enviado por e-mail.

POR QUE A COLUNA SE CHAMA `TOKEN_HASH` E NÃO `TOKEN`

Pelo mesmo motivo que a senha vira `senha_hash`. O token é a chave que troca a senha de uma conta — na prática, vale tanto quanto a senha enquanto está válido. Se ficasse em texto puro e o banco vazasse, quem lesse a tabela poderia trocar a senha de qualquer usuário com pedido pendente. Guardando só o SHA-256, o vazamento não serve para nada: o link já enviado não é recuperável a partir do hash.

Duas colunas fazem o token valer **uma vez só e por tempo limitado**. O `expira_em` é conferido na consulta (`expira_em > NOW()`) e o `usado_em` começa `NULL`, recebendo a data no momento em que a senha é trocada. Um link só funciona enquanto as duas condições valem.

A chave `ON DELETE CASCADE` tem função de limpeza: apagar um usuário apaga junto todos os pedidos de redefinição dele, sem deixar linhas órfãs apontando para um id que não existe mais.

2.6 Conceitos de banco usados no projeto

Conceito	O que faz e onde aparece
PRIMARY KEY	Identificador único de cada linha. Com <code>AUTO_INCREMENT</code> , o banco gera sozinho: 1, 2, 3... Nunca se repete, nem se a linha for apagada.
UNIQUE KEY	Impede valores repetidos. Em <code>usuarios</code> impede dois cadastros com o mesmo e-mail; em <code>medicoes</code> impede duas medições no mesmo dia.
FOREIGN KEY	Liga uma tabela à outra. <code>medicoes.usuario_id</code> tem que apontar para um <code>usuarios.id</code> que existe. O banco recusa uma medição de um usuário inexistente.
ON DELETE CASCADE	Se o usuário for apagado, suas medições são apagadas junto, automaticamente. Evita "dados órfãos" apontando para alguém que não existe mais.
ENUM	Lista fechada de valores permitidos. <code>meta</code> só aceita <code>'perder'</code> , <code>'manter'</code> ou <code>'ganhar'</code> . É uma validação feita pelo próprio banco, além da validação do PHP.
DECIMAL(5,1)	Número com 5 dígitos no total e 1 casa decimal (até 9999,9). Usado para peso e cintura. Não se usa FLOAT para medidas assim porque FLOAT é aproximado e pode gerar valores como 80,000000001.
NULL	Ausência de informação — diferente de zero ou de texto vazio. Cintura <code>NULL</code> significa "não medida", e o gráfico simplesmente não desenha aquele ponto.
utf8mb4	Codificação de caracteres que suporta acentos e cedilha corretamente. O <code>COLLATE utf8mb4_unicode_ci</code> faz a comparação ignorar acentos e maiúsculas — é por isso que buscar "acucar" encontra "Açúcar".
Índice (KEY)	Estrutura auxiliar que acelera buscas numa coluna, como o índice remissivo de um livro. Sem ele, o banco lê todas as linhas.

2.7 Por que o banco cabe em um arquivo só

Todo o banco vive em `banco/nutribalance_completo.sql`: as três tabelas e os 597 alimentos da TACO. Instalar é importar esse arquivo no phpMyAdmin — um passo, sem sequência a respeitar.

Não foi sempre assim. Durante o desenvolvimento, a pasta chegou a ter nove arquivos numerados (`01_schema.sql` até `09_renomear_metas.sql`). Cada um era uma **migration**: uma alteração no formato do banco, com ordem definida. Quando a coluna `peso_alvo` passou a ser necessária, em vez de reescrever a criação da tabela criou-se um arquivo novo que só a adicionava:

```
-- 04_peso_alvo.sql
ALTER TABLE usuarios ADD COLUMN peso_alvo DECIMAL(5,1) NULL AFTER peso_kg;
```

Essa é a prática correta **quando existe um banco em produção com dados dentro**: reescrever a criação da tabela quebraria quem já tinha o banco montado, enquanto a migration é aplicada por cima, preservando os registros.

POR QUE ABANDONAR AS MIGRATIONS AQUI

A condição que justifica migrations nunca se aplicou a este projeto: não há banco em produção, nem outra pessoa com uma cópia de dados para preservar. O único banco existente é o de desenvolvimento, recriável a qualquer momento.

Sem essa condição, a cadeia numerada só acumulou defeitos. Três migrations (`04_peso_alvo` , `05_objetivo` e `09_renomear metas`) acabaram absorvidas pela criação da tabela quando ela foi reescrita, e passaram a **falhar** em banco novo com `Duplicate column name` . A documentação da época orientava ignorar o erro — o que é frágil de defender e transforma a instalação em algo que só funciona para quem já sabe onde ela quebra.

A regra geral: *migrations existem para proteger dados que você não pode perder*. Sem dados a proteger, elas são custo sem benefício, e um schema único e legível vale mais.

O QUE SE PERDEU E O QUE SE GANHOU

Perdeu-se o histórico de evolução do schema registrado em arquivos — mas ele continua no **Git**, que é onde histórico deve morar. Ganhou-se uma instalação de um passo, um schema que se lê de cima a baixo sem reconstruir mentalmente nove alterações, e o fim das credenciais duplicadas: o importador da TACO agora reaproveita a conexão de `site/api/conexao.php` em vez de repetir usuário e senha.

3. O back-end em PHP

Todos os endpoints ficam em `site/api/`. Cada arquivo faz uma coisa só. Nenhum deles gera HTML: todos respondem **JSON**, e quem monta a tela é o JavaScript.

3.1 O bootstrap comum

O arquivo `_bootstrap.php` é incluído por todos os endpoints de conta e perfil. Ele faz três coisas: abre a sessão, define os cabeçalhos de resposta e cria funções auxiliares que evitam repetir código.

```
session_start();
header('Content-Type: application/json; charset=utf-8');
header('Cache-Control: no-store');

/** Envia um JSON e encerra. */
function responder(array $dados, int $status = 200): void {
    http_response_code($status);
    echo json_encode($dados, JSON_UNESCAPED_UNICODE);
    exit;
}

/** Exige método HTTP; encerra com 405 se não bater. */
function exigir_metodo(string $metodo): void {
    if (($_SERVER['REQUEST_METHOD'] ?? '') !== $metodo) {
        responder(['erro' => 'Método não permitido.'], 405);
    }
}

/** Retorna o id do usuário logado ou encerra com 401. */
function exigir_login(): int {
    if (empty($_SESSION['uid'])) {
        responder(['erro' => 'Não autenticado.'], 401);
    }
    return (int) $_SESSION['uid'];
}
```

Vale notar detalhes que parecem pequenos e não são:

- `Cache-Control: no-store` impede o navegador de guardar a resposta em cache. Sem isso, o navegador poderia mostrar dados antigos do perfil depois de uma alteração.
- `JSON_UNESCAPED_UNICODE` faz os acentos saírem como "Almoço" em vez de `Almo\u00e7o`. Funciona igual, mas é legível ao depurar.
- `exit` dentro de `responder()` é essencial: encerra o script na hora, para que nenhum código depois dele execute por engano.

3.2 Conexão com PDO

```
function conectar(): PDO {
    $dsn = 'mysql:host=' . DB_HOST . ';port=' . DB_PORT
        . ';dbname=' . DB_NAME . ';charset=utf8mb4';
    return new PDO($dsn, DB_USER, DB_PASS, [
        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
        PDO::ATTR_EMULATE_PREPARES => false,
    ]);
}
```

PDO (PHP Data Objects) é a interface moderna do PHP para bancos de dados. As três opções configuradas têm motivo:

- `ERRMODE_EXCEPTION` — faz o PDO lançar exceção quando algo dá errado, em vez de falhar em silêncio. É o que permite o `try/catch` capturar o erro e responder 500 com mensagem controlada.
- `FETCH_ASSOC` — as linhas vêm como array associativo (`$linha['nome']`) em vez de virem duplicadas também por posição.
- `EMULATE_PREPARES => false` — força prepared statements *de verdade*, feitos pelo MySQL, e não simulados pelo PHP. É mais seguro, e é o assunto do próximo item.

3.3 Prepared statements e SQL Injection

Este é provavelmente o conceito de segurança mais importante do projeto inteiro.

Imagine que a busca de alimentos fosse escrita concatenando texto:

```
// CODIGO ERRADO - NUNCA FAZER
$sql = "SELECT * FROM alimentos WHERE descricao LIKE '%" . $_GET['q'] . "%'";
```

Se o usuário digitar algo comum, funciona. Mas se digitar:

```
'; DROP TABLE usuarios; --
```

a string final que chega ao banco vira:

```
SELECT * FROM alimentos WHERE descricao LIKE '%'; DROP TABLE usuarios; --'
```

O banco recebe **dois comandos**: a consulta e, em seguida, a destruição da tabela de usuários. Isso é **SQL Injection**. O erro conceitual é que o dado digitado pelo usuário foi tratado como *código*.

O prepared statement resolve separando as duas coisas. Primeiro envia-se a estrutura da consulta com marcadores `?`; depois, os valores. É o que o projeto faz:

```
$stmt = $pdo->prepare('SELECT id, senha_hash, meta_kcal FROM usuarios WHERE email = ?');
$stmt->execute([$email]);
```

POR QUE ISSO FUNCIONA

O banco recebe a consulta já compilada, com um "buraco" onde vai o valor. Quando o valor chega, ele é inserido **como dado**, jamais interpretado como comando. Se o usuário digitar `' ; DROP TABLE usuarios; -`, o banco vai procurar um alimento cuja descrição seja literalmente esse texto — e não achar nada. É a defesa correta, e não depende de o programador lembrar de "limpar" a entrada.

Na busca por múltiplas palavras, o projeto monta os placeholders dinamicamente, mas continua sem concatenar valores:

```
$palavras = preg_split('/\s+/', $q, -1, PREG_SPLIT_NO_EMPTY);
$condicoes = [];
$params = [];
foreach ($palavras as $palavra) {
    $condicoes[] = 'descricao LIKE ?';           // so o marcador entra no SQL
    $params[] = '%' . $palavra . '%';           // o valor vai separado
}
$sql = 'SELECT ... WHERE energia_kcal IS NOT NULL AND '
      . implode(' AND ', $condicoes) . ' ORDER BY descricao LIMIT 30';
$stmt = $pdo->prepare($sql);
$stmt->execute($params);
```

Cada palavra digitada vira um `LIKE`, e todas precisam aparecer na descrição. É por isso que "arroz cozido" encontra "Arroz, integral, cozido", mesmo com uma vírgula e outra palavra no meio.

3.4 Sessões: como o servidor sabe quem está logado

HTTP é um protocolo **sem estado**: cada requisição é independente e o servidor não lembra nada da anterior. Isso levanta um problema: como o servidor sabe que a pessoa que está pedindo o dashboard é a mesma que fez login há cinco minutos?

A resposta é o mecanismo de sessão, que funciona assim:

1. O usuario envia email e senha para `api/login.php`.
2. O PHP confere a senha. Estando correta, executa:

```
session_regenerate_id(true);
$_SESSION['uid'] = 3;
```

Isso cria um ARQUIVO no servidor com o conteudo `{ uid: 3 }`, identificado por um codigo aleatorio longo (o "id de sessao").
3. O PHP envia esse codigo ao navegador dentro de um COOKIE.
4. Em toda requisicao seguinte, o navegador reenvia o cookie automaticamente.
5. O PHP le o cookie, abre o arquivo correspondente no servidor e descobre: `uid = 3`. Agora sabe quem esta falando.

DETALHE DE SEGURANÇA: POR QUE `SESSION_REGENERATE_ID(TRUE)`

Essa linha troca o identificador da sessão no momento do login. Sem ela, o site fica vulnerável a *session fixation*: um atacante forneceria à vítima um identificador de sessão conhecido por ele; quando a vítima fizesse login, aquela mesma sessão passaria a estar autenticada, e o atacante entraria com o identificador que já tinha. Trocando o identificador no login, o antigo deixa de valer.

O importante é que **o cookie guarda apenas o identificador, nunca os dados**. O navegador não sabe o `uid`; ele carrega só uma chave aleatória. Os dados ficam no servidor, fora do alcance do usuário.

O logout desfaz tudo: limpa `$_SESSION`, manda o navegador expirar o cookie (definindo a data no passado) e destrói o arquivo de sessão com `session_destroy()`.

3.5 Senhas: hash, nunca texto puro

A coluna se chama `senha_hash`, e o nome é proposital: **a senha do usuário nunca é guardada**. O que se guarda é o resultado de uma função de hash.

```
// no cadastro (registrar.php)
$hash = password_hash($senha, PASSWORD_DEFAULT);

// no login (login.php)
if (!$u || !password_verify($senha, $u['senha_hash'])) {
    responder(['erro' => 'E-mail ou senha incorretos.'], 401);
}
```

Uma função de hash transforma um texto em outro de tamanho fixo, e o caminho de volta não existe. Da senha se chega ao hash; do hash não se chega à senha. Na verificação, o PHP aplica a mesma função à senha digitada e compara os resultados.

POR QUE ISSO É OBRIGATÓRIO

Se o banco vazar, o atacante leva os hashes, não as senhas. Como muita gente reutiliza senha em vários sites, guardar senha em texto puro exporia as contas de e-mail e banco dos usuários, não só a conta do NutriBalance. Além disso, o `password_hash` aplica um *salt* aleatório automaticamente: dois usuários com a mesma senha têm hashes diferentes, o que impede descobrir todas as contas de uma vez.

Um detalhe deliberado: a mensagem de erro genérica

Quando o login falha, a mensagem é sempre "E-mail ou senha incorretos" — nunca "esse e-mail não existe" ou "senha errada". Isso é intencional. Mensagens específicas permitiriam a um atacante descobrir quais e-mails têm conta no site, testando um por um. O código deixa isso explícito num comentário: *"Mensagem genérica (não revela se o e-mail existe)"*.

3.6 Códigos de status HTTP

Toda resposta HTTP carrega um número que diz o que aconteceu. O projeto usa esses códigos de forma consistente, e o JavaScript decide o que fazer com base neles.

Código	Significado	Onde o projeto usa
200	OK	Sucesso em qualquer endpoint.
401	Não autenticado	Sem sessão válida, ou senha incorreta. O front-end redireciona para o login.
405	Método não permitido	Ex.: acessar por GET um endpoint que exige POST.
409	Conflito	Cadastro com e-mail que já existe.
422	Dado inválido	Idade fora de 10–120, peso fora de 30–350, sexo não informado.
500	Erro do servidor	Falha inesperada, como o banco fora do ar.

POR QUE 422 E NÃO 400

400 significa "requisição malformada" — o servidor não conseguiu nem entender. 422 significa "entendi perfeitamente, mas o conteúdo não passa nas regras de negócio". Uma idade de 500 anos é sintaticamente válida e semanticamente impossível: é exatamente o caso do 422.

3.7 Percurso de cada endpoint

Arquivo	O que faz
<code>_bootstrap.php</code>	Base comum: sessão, cabeçalhos e as funções <code>responder</code> , <code>corpo</code> , <code>exigir_metodo</code> , <code>exigir_login</code> .
<code>conexao.php</code>	Credenciais e a função <code>conectar()</code> que devolve o PDO.
<code>registrar.php</code>	Valida nome, e-mail e senha (mínimo 6 caracteres); verifica se o e-mail já existe (409); grava com hash; abre a sessão.
<code>login.php</code>	Confere a senha com <code>password_verify</code> ; regenera o id de sessão; devolve <code>perfil_completo</code> para o front decidir o destino.
<code>logout.php</code>	Limpa a sessão, expira o cookie e destrói o arquivo de sessão.
<code>sessao.php</code>	Guarda das páginas. Sem sessão devolve 401; com sessão devolve o usuário e o perfil no formato camelCase que o JavaScript espera.
<code>salvar_perfil.php</code>	Recebe os dados do onboarding, valida faixas, recalcula a meta e os macros, grava, e semeia a primeira medição de hoje.
<code>buscar.php</code>	Busca na TACO por palavras, devolve até 30 resultados no formato do site.
<code>salvar_medicao.php</code>	Upsert da medição do dia; sincroniza <code>usuarios.peso_kg</code> com a medição mais recente e recalcula as metas.
<code>listar_medicoes.php</code>	Devolve o histórico em ordem cronológica, para o gráfico.
<code>foto.php</code>	Envio, troca e remoção da foto de perfil.
<code>esqueceu_senha.php</code>	Gera o token de redefinição, grava o hash dele e envia o link por e-mail. Responde sempre a mesma coisa, exista ou não a conta.
<code>redefinir_senha.php</code>	Confere o token (válido, não usado, no prazo) e grava a senha nova com hash, dentro de uma transação.
<code>email.php</code>	Infraestrutura, não é endpoint: envia mensagens pelo SMTP usando o PHPMailer.

3.8 Por que o servidor recalcula tudo

O cabeçalho do `salvar_perfil.php` declara a regra explicitamente: *"Valida as faixas, RECALCULA meta_kcal + macros aqui (autoridade — não confia no valor que o front mandou)"*.

A fórmula de cálculo existe em dois lugares: em `js/calculo.js` e em `api/salvar_perfil.php`. Isso é duplicação de código, o que normalmente se evita. Aqui é deliberado, e a justificativa é sólida:

- A versão em **JavaScript** existe para a experiência de uso: ela atualiza a prévia na tela conforme o usuário digita, sem precisar ir ao servidor a cada tecla.
- A versão em **PHP** é a que vale. É ela que grava no banco.

Se o usuário adulterar o JavaScript, ele consegue mudar o número que aparece na tela dele — e nada mais. Ao salvar, o servidor refaz a conta a partir de sexo, idade, altura, peso, atividade e meta, e grava o

próprio resultado.

PONTO DE ATENÇÃO HONESTO

Código duplicado é uma dívida real: se a fórmula mudar, é preciso alterar nos dois arquivos, e esquecer um deles causaria divergência entre a prévia e o valor salvo. A alternativa seria calcular só no servidor e consultar a cada mudança de campo, ao custo de muitas requisições. A escolha foi consciente, e está registrada em comentário nos dois arquivos justamente para que ninguém altere um sem o outro.

3.9 Upload de imagem com validação real

O `foto.php` aceita a imagem e faz quatro verificações antes de gravar:

```
const TAMANHO_MAX    = 3 * 1024 * 1024; // 3 MB
const TIPOS_ACEITOS = [
    'image/jpeg' => 'jpg',
    'image/png'   => 'png',
    'image/webp'  => 'webp',
];

// Valida que e mesmo uma imagem e descobre o tipo real
// (nao confia na extensao).
$info = @getimagesize($arquivo['tmp_name']);
$mime = $info['mime'] ?? '';
if (!$info || !isset(TIPOS_ACEITOS[$mime])) {
    responder(['erro' => 'Formato inválido. Use JPG, PNG ou WEBP.'], 422);
}
$ext = TIPOS_ACEITOS[$mime];
```

POR QUE NÃO OLHAR A EXTENSÃO DO ARQUIVO

Qualquer pessoa pode renomear `virus.php` para `foto.jpg`. A extensão é só texto no nome, não diz nada sobre o conteúdo. A função `getimagesize()` abre o arquivo e lê os primeiros bytes, onde ficam as assinaturas que identificam o formato real. Se não for uma imagem de verdade, ela falha. O nome final gravado no disco é gerado pelo próprio servidor (`foto_<id>_<hash aleatório>.<ext>`), então o nome enviado pelo usuário é descartado por completo.

A ordem das operações também importa: o arquivo novo é gravado **primeiro** e o antigo só é apagado **depois** que o novo já está no disco e o banco atualizado. Se algo falhar no meio, o usuário fica com a foto antiga em vez de ficar sem nenhuma.

3.10 Recuperação de senha: token de uso único

O fluxo de "esqueci minha senha" tem um problema central: como deixar alguém trocar a senha *sem* saber a senha atual, e ainda assim ter certeza de que é o dono da conta. A resposta é usar o acesso ao e-mail como prova de identidade.

O caminho ingênuo seria gerar uma senha nova e enviá-la pronta por e-mail. O projeto não faz isso, por dois motivos: a senha trafegaria em texto puro e ficaria guardada na caixa de entrada indefinidamente; e qualquer pessoa poderia trocar a senha de outra só sabendo o e-mail dela, mesmo sem acesso à caixa — bastaria pedir a redefinição para trancar a vítima fora da conta.

Em vez disso, o sistema emite um **token**: uma chave aleatória, temporária, que autoriza uma única troca de senha. A senha atual continua valendo até a pessoa efetivamente escolher a nova.

Os dois passos

```
// 1. esqueceu_senha.php – emite o token
$token = bin2hex(random_bytes(32));           // 64 caracteres imprevisíveis
$pdo->prepare(
    'INSERT INTO redefinicoes_senha (usuario_id, token_hash, expira_em)
    VALUES (?, ?, DATE_ADD(NOW(), INTERVAL 60 MINUTE))'
)->execute([$usuario['id'], hash('sha256', $token)]);

$link = url_site() . '/redefinir-senha.html?token=' . $token;

// 2. redefinir_senha.php – consome o token
$stmt = $pdo->prepare(
    'SELECT id, usuario_id FROM redefinicoes_senha
    WHERE token_hash = ? AND usado_em IS NULL AND expira_em > NOW()'
);
$stmt->execute([hash('sha256', $token)]);
```

O token em claro existe em dois lugares apenas: no e-mail da pessoa e na barra de endereço do navegador dela. No banco fica só o hash. Na hora de validar, o servidor aplica o mesmo `sha256` ao token recebido e procura por igualdade — a mesma lógica do `password_verify`, na seção 3.5.

POR QUE RANDOM_BYTES E NÃO MD5(TIME())

Um token precisa ser **imprevisível**. O `time()` devolve o segundo atual: quem soubesse aproximadamente quando o pedido foi feito testaria algumas centenas de valores e acertaria o token. A função `random_bytes()` usa o gerador criptográfico do sistema operacional; com 32 bytes, o espaço de possibilidades é grande demais para ser varrido por tentativa e erro.

Três proteções que valem mencionar na defesa

Resposta genérica. O endpoint responde exatamente a mesma mensagem se o e-mail existe ou não. Se dissesse "e-mail não cadastrado", a tela viraria uma ferramenta para descobrir quem tem conta no site — o mesmo critério já aplicado no `login.php`, que nunca diz se errou o e-mail ou a senha.

Uso único e prazo. Ao trocar a senha, o `usado_em` é preenchido e o link morre. Pedir um link novo também invalida os anteriores, então só o último e-mail funciona. Fora do prazo de uma hora, a consulta não encontra nada.

Transação. Gravar a senha nova e dar baixa no token são duas operações que precisam valer juntas:

```
$pdo->beginTransaction();
$pdo->prepare('UPDATE usuarios SET senha_hash = ? WHERE id = ?')
->execute([password_hash($senha, PASSWORD_DEFAULT), $pedido['usuario_id']]);
$pdo->prepare('UPDATE redefinicoes_senha SET usado_em = NOW() WHERE id = ?')
->execute([$pedido['id']]);
$pdo->commit();
```

Sem a transação, uma falha entre os dois comandos deixaria a senha já trocada com o link ainda ativo — utilizável de novo por quem tivesse o e-mail. Com ela, ou as duas gravações acontecem, ou nenhuma.

DEPENDÊNCIA EXTERNA

Este é o único ponto do projeto que depende de um serviço de fora: o envio precisa de um servidor SMTP. A função `mail()` nativa do PHP não serve, porque o XAMPP não inclui servidor de e-mail — ela falha silenciosamente. O projeto usa o **PHPMailer**, instalado como três arquivos em `api/lib/PHPMailer/`, sem Composer. É a única biblioteca externa do NutriBalance. As credenciais ficam em `email_config.php`, fora do controle de versão.

4. A matemática nutricional

Esta é a parte que a banca mais provavelmente vai questionar, porque é onde o projeto deixa de ser "um site" e passa a ter uma regra de negócio própria. Toda a lógica está em `js/calculo.js`, espelhada em `api/salvar_perfil.php`.

4.1 Taxa metabólica basal (Mifflin-St Jeor)

A **TMB** é quanto de energia o corpo gasta em repouso absoluto, só para se manter vivo: respiração, circulação, temperatura, funcionamento dos órgãos. É o piso do gasto diário.

O projeto usa a equação de **Mifflin-St Jeor**, publicada em 1990:

```
Homens:    TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × idade) + 5
Mulheres:  TMB = (10 × peso_kg) + (6,25 × altura_cm) - (5 × idade) - 161
```

No código:

```
const tmb = 10 * peso + 6.25 * altura - 5 * idade + (sexo === "M" ? 5 : -161);
```

POR QUE MIFFLIN-ST JEOR E NÃO HARRIS-BENEDICT

Harris-Benedict é mais antiga (1919, revisada em 1984) e é a fórmula que aparece na maioria dos sites. Mifflin-St Jeor foi desenvolvida depois, com uma amostra mais representativa da população contemporânea, e estudos de validação a apontam como mais precisa para pessoas com sobrepeso e obesidade — justamente o público que mais usa um app de contagem de calorias. A Academy of Nutrition and Dietetics dos Estados Unidos a recomenda como padrão para adultos saudáveis.

Sobre os termos da fórmula, vale entender o sinal de cada um:

- **Peso e altura entram positivo:** mais massa corporal significa mais tecido para manter, logo mais energia.
- **Idade entra negativo:** o metabolismo desacelera com o tempo, entre outros fatores pela perda gradual de massa magra.
- **A constante muda com o sexo (+5 ou -161):** em média, homens têm proporção maior de massa magra, que é metabolicamente mais ativa que a gordura.

4.2 Fator de atividade e TDEE

A TMB é o gasto em repouso. Ninguém fica em repouso o dia inteiro. O **TDEE** (*Total Daily Energy Expenditure*, gasto energético diário total) é a TMB multiplicada por um fator que representa o nível de atividade:

```
const FATOR_ATIVIDADE = {
  sedentario: 1.2,    // pouco ou nenhum exercicio
  leve: 1.375,    // exercicio leve, 1 a 3 dias por semana
  moderado: 1.55,    // exercicio moderado, 3 a 5 dias
  intenso: 1.725,    // exercicio pesado, 6 a 7 dias
  muito_intenso: 1.9,    // trabalho fisico pesado ou 2 treinos por dia
};

TDEE = TMB × fator
```

Um sedentário gasta cerca de 20% acima do basal só com as atividades cotidianas — andar, sentar, digerir. Alguém muito ativo pode chegar a 90% acima.

4.3 Déficit e superávit calórico

Este é o princípio central do sistema, e ele é simples: **o peso corporal responde ao balanço entre a energia consumida e a energia gasta.**

- Consumir **menos** que o TDEE cria um **déficit**. O corpo cobre a diferença usando reservas — resultado: perda de peso.
- Consumir **mais** que o TDEE cria um **superávit**. O excedente é armazenado — resultado: ganho de peso.
- Consumir **igual** ao TDEE mantém o peso.

```
const AJUSTE_META = {
  perder: -0.15,    // deficit de 15%
  manter: 0,
  ganhar: 0.15,    // superavit de 15%
};

meta_kcal = TDEE × (1 + ajuste)
```

POR QUE 15%, E NÃO 30%

Um déficit maior emagreceria mais rápido, então por que não usar? Porque déficits agressivos têm três problemas: aumentam a perda de massa magra junto com a gordura, são difíceis de sustentar (levando ao abandono), e podem gerar deficiência de nutrientes por reduzir demais o volume de comida. Um déficit percentual de 10% a 20% é a faixa geralmente recomendada como sustentável. O projeto adota 15%, o meio dessa faixa.

Usar **percentual** em vez de um valor fixo (como "menos 500 kcal") também é uma escolha: 500 kcal a menos representa muito mais para quem gasta 1800 do que para quem gasta 3500. O percentual se ajusta sozinho ao tamanho da pessoa.

4.4 Macronutrientes e os fatores 4/4/9

Não basta saber quantas calorias. Os **macronutrientes** — carboidrato, proteína e gordura — têm papéis diferentes no corpo, e cada grama fornece uma quantidade específica de energia:

Macronutriente	kcal por grama	Papel principal
Carboidrato	4 kcal/g	Fonte primária de energia rápida
Proteína	4 kcal/g	Construção e manutenção de tecidos, inclusive músculo
Gordura	9 kcal/g	Reserva de energia, hormônios, absorção de vitaminas

Esses são os **fatores de Atwater**, estabelecidos no fim do século XIX e usados até hoje em rotulagem nutricional. A gordura fornece mais que o dobro por grama — é por isso que alimentos gordurosos são caloricamente densos.

A conversão de calorias para gramas é uma divisão:

```
gramas_de_carboidrato = (meta_kcal × percentual_carbo) / 4
gramas_de_proteina    = (meta_kcal × percentual_proteina) / 4
gramas_de_gordura     = (meta_kcal × percentual_gordura) / 9
```

4.5 As divisões 50/20/30 e 40/35/25

```
const DIVISAO_PADRAO    = { carbo: 0.5, proteina: 0.2, gordura: 0.3 };
const DIVISAO_DEFINICAO = { carbo: 0.4, proteina: 0.35, gordura: 0.25 };
```

A divisão padrão (50% carboidrato, 20% proteína, 30% gordura) está dentro das faixas de distribuição aceitáveis de macronutrientes recomendadas por órgãos de nutrição, e serve para a maioria dos objetivos.

A segunda divisão entra em um caso específico: quando o usuário escolhe **manter** o peso mas com objetivo **definir** — o que em treinamento se chama *recomposição corporal*: perder gordura e ganhar músculo ao mesmo tempo, sem que a balança mude muito.

POR QUE MAIS PROTEÍNA NA RECOMPOSIÇÃO

A proteína fornece os aminoácidos que o corpo usa para construir e reparar músculo. Em um contexto de recomposição, uma ingestão proteica mais alta ajuda a preservar e ganhar massa magra enquanto a gordura é reduzida. Por isso a proteína sobe de 20% para 35%, e carboidrato e gordura cedem espaço proporcionalmente.

A função que preserva a proporção do usuário

O modal de perfil permite ajustar a meta de calorias manualmente. Quando isso acontece, os macros precisam ser recalculados — mas seria errado jogar de volta para 50/20/30, porque o usuário pode ter uma divisão personalizada. A função `ajustarMacrosParaKcal` resolve isso mantendo a proporção atual:

```
function ajustarMacrosParaKcal(novaKcal, macros) {
  const k = Number(novaKcal) || 0;
  const kcalCarbo = (Number(macros.metaCarbo) || 0) * 4;
  const kcalProteina = (Number(macros.metaProteina) || 0) * 4;
  const kcalGordura = (Number(macros.metaGordura) || 0) * 9;
  const total = kcalCarbo + kcalProteina + kcalGordura;
  if (!(total > 0)) return derivarMetasMacro(k); // sem base valida, usa o padrao
  return {
    metaCarbo: Math.round((k * (kcalCarbo / total)) / 4),
    metaProteina: Math.round((k * (kcalProteina / total)) / 4),
    metaGordura: Math.round((k * (kcalGordura / total)) / 9),
  };
}
```

O raciocínio: converte os gramas atuais de volta para calorias, descobre qual fração do total cada macro representa, aplica essas mesmas frações à nova meta e converte de volta para gramas.

4.6 IMC e faixa de peso ideal

O **Índice de Massa Corporal** relaciona peso e altura:

$$\text{IMC} = \text{peso_kg} / (\text{altura_m})^2$$

A classificação segue as faixas da Organização Mundial da Saúde:

IMC	Classificação
abaixo de 18,5	Abaixo do peso
18,5 a 24,9	Peso normal
25 a 29,9	Sobrepeso
30 ou mais	Obesidade

A faixa de peso ideal é o cálculo inverso: fixa-se o IMC nos limites saudáveis e resolve-se para o peso.

$$\begin{aligned} \text{peso_minimo} &= 18,5 \times (\text{altura_m})^2 \\ \text{peso_maximo} &= 24,9 \times (\text{altura_m})^2 \end{aligned}$$

LIMITAÇÃO QUE VOCÊ DEVE SABER APONTAR

O IMC não distingue massa magra de gordura. Um atleta musculoso pode ser classificado como "sobrepeso" tendo pouquíssima gordura corporal. É um indicador populacional, útil como referência rápida, não um diagnóstico individual. Foi justamente por isso que o projeto incluiu o acompanhamento da **cintura**: ela reflete mudanças de composição corporal que o peso sozinho esconde — é possível manter o peso e reduzir a cintura, sinal de troca de gordura por músculo.

4.7 Previsão de prazo até o objetivo

A função `preverResultado()` estima quando o usuário alcança o peso desejado. Ela parte de uma constante:

```
const KCAL_POR_KG = 7700;
```

A ideia é que aproximadamente 7.700 kcal correspondem a 1 kg de tecido corporal. Daí o cálculo é direto:

```
diferencaDia = |TDEE - meta_kcal|           // deficit ou superavit por dia
dias          = (kg_a_percorrer x 7700) / diferencaDia
taxaSemanas   = (diferencaDia x 7) / 7700    // kg por semana
```

O sistema ainda emite um alerta quando o ritmo é agressivo:

```
ritmoAcelerado: taxaSemana > peso * 0.01    // mais de 1% do peso por semana
```

LIMITAÇÕES DESTA PREVISÃO — IMPORTANTE ADMITIR

O modelo é **linear**, e a realidade não é. Três limitações concretas:

- **O TDEE muda conforme o peso muda.** Ao emagrecer, a pessoa fica mais leve, a TMB cai e o déficit real diminui — o emagrecimento desacelera. O cálculo assume o TDEE inicial constante, então tende a ser otimista em prazos longos.
- **O valor 7.700 é uma aproximação.** A literatura cita entre 7.000 e 7.700, e ele descreve tecido adiposo; o peso perdido inclui também água e massa magra.
- **Adesão não é considerada.** O cálculo pressupõe que a meta será cumprida exatamente todos os dias.

O próprio código registra isso em comentário: *"Modelo linear (ritmo constante) — o TDEE real muda com o peso, então trate como estimativa."* Saber apontar a limitação da própria ferramenta é mais forte, numa defesa, do que fingir precisão que ela não tem.

4.8 Exemplo numérico completo

Acompanhe um caso do início ao fim. Estes são valores reais, conferidos no sistema em execução.

Dados de entrada: homem, 17 anos, 175 cm, 80 kg, atividade moderada, meta perder, peso desejado 72 kg, objetivo manter (sem recomposição).

PASSO 1 – Taxa metabolica basal (Mifflin-St Jeor, homem)

$$\begin{aligned} \text{TMB} &= (10 \times 80) + (6,25 \times 175) - (5 \times 17) + 5 \\ &= 800 + 1093,75 - 85 + 5 \\ &= 1813,75 \text{ kcal/dia} \end{aligned}$$

PASSO 2 – Gasto diario total (atividade moderada, fator 1,55)

$$\begin{aligned} \text{TDEE} &= 1813,75 \times 1,55 \\ &= 2811,31 \text{ kcal/dia} \end{aligned}$$

PASSO 3 – Meta com deficit de 15%

$$\begin{aligned} \text{meta_kcal} &= 2811,31 \times (1 - 0,15) \\ &= 2811,31 \times 0,85 \\ &= 2389,61 \rightarrow \text{arredondado: } 2390 \text{ kcal/dia} \end{aligned}$$

PASSO 4 – Macronutrientes (divisao padrao 50/20/30)

$$\begin{aligned} \text{carboidrato} &= (2390 \times 0,50) / 4 = 1195 / 4 = 298,75 \rightarrow 299 \text{ g} \\ \text{proteina} &= (2390 \times 0,20) / 4 = 478 / 4 = 119,50 \rightarrow 120 \text{ g} \\ \text{gordura} &= (2390 \times 0,30) / 9 = 717 / 9 = 79,67 \rightarrow 80 \text{ g} \end{aligned}$$

$$\begin{aligned} \text{Conferencia: } &(299 \times 4) + (120 \times 4) + (80 \times 9) \\ &= 1196 + 480 + 720 = 2396 \text{ kcal} \end{aligned}$$

(a pequena diferenca para 2390 vem do arredondamento de cada macro)

PASSO 5 – IMC e faixa saudavel

$$\begin{aligned} \text{altura em metros} &= 1,75 \rightarrow 1,75^2 = 3,0625 \\ \text{IMC} &= 80 / 3,0625 = 26,12 \rightarrow \text{"Sobrepeso"} \\ \text{peso minimo} &= 18,5 \times 3,0625 = 56,7 \text{ kg} \\ \text{peso maximo} &= 24,9 \times 3,0625 = 76,3 \text{ kg} \end{aligned}$$

PASSO 6 – Previsao de prazo

$$\begin{aligned} \text{kg a perder} &= 80 - 72 = 8 \text{ kg} \\ \text{deficit/dia} &= |2811,31 - 2390| = 421,31 \text{ kcal} \\ \text{dias} &= (8 \times 7700) / 421,31 = 61600 / 421,31 = 146,2 \text{ dias} \\ \text{taxa/semana} &= (421,31 \times 7) / 7700 = 0,38 \text{ kg por semana} \end{aligned}$$

Como 146 dias $\geq$ 56, o texto sai em meses:

$$146,2 / 30 = 4,87 \rightarrow \text{arredondado: } 5$$

Exibido na tela: "Meta daqui a 5 meses"

Ritmo acelerado? $0,38 > (80 \times 0,01 = 0,8)$? Nao.

Logo, nenhum alerta e mostrado.

COMO VALIDAR ISSO NA APRESENTAÇÃO

Se a banca pedir para você provar que o sistema calcula certo, este é o roteiro: informe esses dados na tela de personalização e confira que aparecem 2.390 kcal, os macros 299/120/80, o IMC 26,1 classificado como "Sobrepeso", a faixa de 56,7 a 76,3 kg e o prazo de 5 meses. Todos os números acima foram verificados no sistema rodando.

5. O front-end

5.1 Por que HTML, CSS e JavaScript puros

O projeto não usa React, Vue, jQuery nem qualquer biblioteca. Isso foi decisão, não limitação, e há bons argumentos para defendê-la:

- **Nada a instalar.** Não há `node_modules`, nem etapa de build. Copiar a pasta e abrir pelo Apache basta — o que importa muito para apresentar em um computador que não é o seu.
- **Aprendizado real.** Escrever a manipulação de DOM na mão ensina o que os frameworks fazem por baixo. Quem aprende só o framework não sabe explicar o que está acontecendo.
- **Tamanho adequado ao problema.** O site tem quatro páginas. Um framework resolveria problemas que este projeto não tem, ao custo de complexidade que ele não precisa.

5.2 Módulos ES: cada arquivo declara do que depende

O JavaScript do site é organizado em **módulos ES**. Cada arquivo declara explicitamente o que oferece (`export`) e o que consome (`import`). O HTML carrega um único script — o resto o navegador resolve sozinho, seguindo os imports:

```
<!-- index.html -->
<script type="module" src="js/principal.js"></script>
```

```
// js/tela.js – o topo do arquivo diz tudo de que ele precisa
import { estado } from "../estado.js";
import { arred, escaparHtml, rotuloData } from "../utilitarios.js";
import { blocoMacros, totaisDoDia } from "../macros.js";
```

Arquivo	Responsabilidade
<code>principal.js</code>	<i>Entry point</i> do dashboard. Valida a sessão, carrega os dados e monta a tela.
<code>estado.js</code>	O estado compartilhado do dashboard: dados do usuário, dia exibido, previsão.
<code>dados.js</code>	<code>localStorage</code> e a migração entre versões do formato salvo.
<code>utilitarios.js</code>	Datas, texto e ids. Não importa nada — é a base de todos.
<code>macros.js</code>	Converte gramas em kcal e macros consumidos.
<code>tela.js</code>	Desenha o dashboard. Só lê o estado; nunca o altera.
<code>refeicoes.js</code> , <code>alimentos.js</code> , <code>perfil.js</code>	As três áreas de interação. Alteram o estado e chamam <code>atualizarTela()</code> .
<code>calculo.js</code>	Só matemática: TMB, TDEE, macros, IMC, previsão. Não toca na tela nem na rede. É o arquivo mais fácil de testar e reaproveitar.
<code>auth.js</code>	Login, cadastro, logout e a guarda de sessão. Usado por todas as páginas.
<code>progresso.js</code>	Histórico de peso e cintura, e o gráfico SVG.
<code>personalizacao.js</code>	Só na tela de onboarding: lê os campos e mostra a prévia ao vivo.
<code>senha.js</code>	O botão de olho que mostra e oculta os campos de senha.
<code>travas.js</code>	Bloqueia os atalhos de zoom (teclado, <code>Ctrl</code> + roda, pinça no trackpad). A trava de seleção de texto e a de pinça no toque ficam no <code>base.css</code> , que resolve as duas sem JavaScript.

O QUE ISSO RESOLVEU

Antes, cada arquivo era uma tag `<script>` separada e todas as funções viravam **globais**. Funcionava, mas com dois problemas. A **ordem** das tags era significativa e invisível: trocar duas linhas de lugar quebrava o site sem erro claro. E as dependências eram **invisíveis** — abrindo um arquivo, não havia como saber que ele precisava de uma função definida em outro; só descobrindo em execução.

Com módulos, o topo de cada arquivo é a lista de dependências. Um nome escrito errado vira erro imediato no console, não um `undefined` silencioso três telas adiante. E o escopo é privado por padrão: o que não é exportado não existe para os outros arquivos, sem precisar de truques como a IIFE que o `progresso.js` usava.

O estado compartilhado

Módulos trazem uma regra que muda como o estado é escrito: quem importa uma variável só pode **lê-la**. Reatribuir de outro arquivo é erro. Como vários módulos precisam alterar o dia exibido ou os dados do usuário, esses valores vivem como campos de um objeto:

```
// js/estado.js
export const estado = {
  dados: null,           // refeicoes e perfil do usuario logado
  previsao: null,        // prazo estimado ate o objetivo
  dataSelecionada: isoHoje(), // dia que o dashboard esta exibindo
  chaveStorage: null,    // chave do localStorage deste usuario
};
```

A referência ao objeto nunca muda, então todos os módulos enxergam a mesma coisa; o que muda são os campos. Quando `refeicoes.js` faz `estado.dataSelecionada = ...`, o `tela.js` lê o valor novo na próxima chamada.

DETALHE DE EXECUÇÃO

Um `<script type="module">` é adiado automaticamente: só roda depois que o HTML terminou de ser lido. Por isso os módulos podem buscar elementos da página no topo do arquivo, sem esperar por evento nenhum. Em compensação, módulos só carregam por `http://` — abrir o `index.html` por clique duplo não funciona, o que já era verdade por causa do PHP.

5.3 A guarda de sessão

Nenhuma página do sistema deve ser acessível sem login. Quem garante isso é a função `exigirSessao()`, em `auth.js`:

```
async function exigirSessao(opcoes) {
  const permitirIncompleto = opcoes && opcoes.permitirIncompleto;
  let res;
  try {
    res = await fetch("api/sessao.php", { headers: { Accept: "application/json" } });
  } catch {
    window.location.href = "login.html"; // servidor fora do ar
    return null;
  }
  if (res.status === 401) { // sem sessao
    window.location.href = "login.html";
    return null;
  }
  const dados = await res.json().catch(() => null);
  if (!dados || !dados.ok) {
    window.location.href = "login.html";
    return null;
  }
  if (!dados.perfil_completo && !permitirIncompleto) {
    window.location.href = "personalizacao.html"; // falta o onboarding
    return null;
  }
  return dados;
}
```

Ela cobre quatro situações e as trata de forma diferente: servidor inacessível, sem sessão, resposta inválida e perfil incompleto. O parâmetro `permitirIncompleto` existe para um caso específico: a própria

tela de personalização precisa aceitar perfil incompleto — é lá que ele se completa. Sem essa exceção, o site entraria em laço de redirecionamento infinito.

PONTO CONCEITUAL IMPORTANTE

Essa guarda é de **usabilidade**, não de segurança. Ela roda no navegador e pode ser burlada. A segurança de verdade está no back-end: todo endpoint chama `exigir_login()` e devolve 401 sem sessão válida. Mesmo que alguém force a abertura do `index.html`, a tela apareceria vazia — nenhum dado viria do servidor. Vale muito saber fazer essa distinção numa defesa: *validação no cliente é conveniência; validação no servidor é segurança*.

5.4 Renderização com template strings

O HTML da lista de refeições não existe no arquivo `index.html`; ele é gerado por JavaScript a partir dos dados. A técnica usada é a **template string** — texto entre crases, onde `${...}` insere valores:

```
function linhaProgresso(rotulo, unidade, consumido, meta, classe) {
  const pct = meta > 0 ? Math.min(100, (consumido / meta) * 100) : 0;
  const passou = consumido > meta && meta > 0;
  const restante = Math.max(0, meta - consumido);
  return `
    <div class="progresso-linha ${classe} ${passou ? "passou" : ""}">
      <div class="progresso-topo">
        <span class="progresso-rotulo">${rotulo}</span>
        <span class="progresso-numeros">${arred(consumido)} / ${arred(meta)} ${unidade}</span>
      </div>
      <div class="progresso-trilha">
        <div class="progresso-barra" style="width:${pct}%"></div>
      </div>
    `
  ...
}
```

Dois detalhes de lógica valem notar. O `Math.min(100, ...)` impede que a barra ultrapasse visualmente a largura do container quando o usuário come mais que a meta. E a classe condicional `${passou ? "passou" : ""}` adiciona uma classe extra que o CSS usa para pintar a barra de vermelho — a decisão visual fica no CSS, o JavaScript só sinaliza o estado.

5.5 Delegação de eventos

Cada refeição na tela tem botões de remover item, remover refeição e adicionar alimento. Como o HTML é regenerado toda vez que algo muda, associar um `addEventListener` a cada botão daria trabalho e criaria vazamento de memória.

A solução é **delegação**: um único ouvinte no elemento pai, que descobre qual botão foi clicado.


```
document.getElementById("lista-refeicoes").addEventListener("click", (e) => {
  // dataset converte o atributo kebab-case em camelCase:
  // data-adicionar-alimento -> dataset.adicionarAlimento
  const botaoAdicionarAlimento = e.target.closest("[data-adicionar-alimento]");
  if (botaoAdicionarAlimento) return
  abrirBuscaAlimento(botaoAdicionarAlimento.dataset.adicionarAlimento);

  const botaoRemoverItem = e.target.closest("[data-remover-item]");
  if (botaoRemoverItem) return removerItem(botaoRemoverItem.dataset.removerItem);

  const botaoRemoverRefeicao = e.target.closest("[data-remover-refeicao]");
  if (botaoRemoverRefeicao && confirm("Remover essa refeição inteira?")) {
    return removerRefeicao(botaoRemoverRefeicao.dataset.removerRefeicao);
  }
});
```

Funciona porque o clique **borbulha** (*event bubbling*): ao clicar num botão, o evento sobe pelos elementos pais até chegar ao container. Lá, `e.target` é o elemento efetivamente clicado, e `closest()` sobe a partir dele procurando o primeiro ancestral que tenha o atributo indicado.

O identificador da refeição viaja pelo próprio HTML, em um atributo `data-*`. Essa é a forma padrão de anexar dados a um elemento sem inventar atributos inválidos.

A VANTAGEM DECISIVA

O ouvinte é registrado **uma vez**, na carga da página, e continua funcionando para botões que ainda nem existiam quando ele foi criado. Como a lista é reconstruída a cada alteração, isso é essencial — sem delegação, seria preciso reassociar todos os eventos após cada renderização.

5.6 XSS e a função `escaparHtml`

Quando o texto do usuário é inserido na página com `innerHTML`, ele é interpretado como HTML. Se alguém cadastrar um alimento chamado:

```
<img src=x onerror="alert('invadido')">
```

esse código seria **executado** no navegador de quem visse a lista. Isso é **XSS** (*Cross-Site Scripting*). Um ataque real roubaria o cookie de sessão e enviaria para o atacante.

A defesa do projeto é converter os caracteres especiais antes de inserir:

```
function escaparHtml(texto) {
  const div = document.createElement("div");
  div.textContent = texto ?? "";
  return div.innerHTML;
}
```

POR QUE ESSA IMPLEMENTAÇÃO É ELEGANTE

Ela não usa expressão regular nem lista de caracteres a substituir — que é onde normalmente se esquece um caso. Ela cria uma `<div>` na memória e atribui o texto via `textContent`, que trata tudo como **texto puro**. Ao ler de volta `innerHTML`, o próprio navegador devolve a versão escapada: `< virou <`, `> virou >`. Delega ao navegador uma tarefa que ele já sabe fazer perfeitamente.

Por isso todo dado vindo do usuário passa por ela antes de entrar no HTML: `escaparHtml(item.nome)`, `escaparHtml(refeicao.tipo)`, `escaparHtml(a.nome)`.

5.7 Debounce na busca

Se cada tecla digitada disparasse uma requisição, digitar "arroz" geraria cinco chamadas ao servidor — quatro delas inúteis, porque o usuário ainda não terminou. O **debounce** resolve adiando a execução até que as teclas parem:

```
function atrasarChamada(fn, ms) {
  let t;
  return (...args) => {
    clearTimeout(t); // cancela o agendamento anterior
    t = setTimeout(() => fn(...args), ms); // e agenda de novo
  };
}

const executarBusca = atrasarChamada(async () => { ... }, 400);
campoBuscaAlimento.addEventListener("input", executarBusca);
```

A cada tecla, o temporizador anterior é cancelado e um novo é criado. A busca só roda quando o usuário fica 400 ms sem digitar. Digitar "arroz" gera uma requisição em vez de cinco.

CONCEITO POR TRÁS: CLOSURE

A função devolvida "lembra" da variável `t` mesmo depois que `atrasarChamada` já terminou de executar. Isso se chama **closure**: uma função que mantém acesso ao escopo onde foi criada. É o que permite que cada chamada saiba qual temporizador cancelar.

5.8 O gráfico SVG feito à mão

O gráfico de evolução de peso e cintura não usa biblioteca. Ele é **SVG** gerado por código, e entender essa parte impressiona numa defesa, porque envolve matemática de conversão de escala.

O problema: os dados estão em quilogramas e datas; a tela está em pixels. É preciso mapear um espaço no outro.

```

const LARGURA = 300, ALTURA = 132;
const margemEsq = 10, margemDir = 12, margemTopo = 14, margemBase = 22;

// Folga vertical: 15% da amplitude (ou 1 unidade quando todos iguais).
const folga = vmax - vmin > 0 ? (vmax - vmin) * 0.15 : 1;
vmin -= folga;
vmax += folga;

// eixo X: data -> pixel
const x = (iso) => {
  if (t1 === t0) return (margemEsq + (LARGURA - margemDir)) / 2; // ponto unico: centro
  const t = new Date(iso + "T00:00:00").getTime();
  return margemEsq + ((t - t0) / (t1 - t0)) * (LARGURA - margemEsq - margemDir);
};

// eixo Y: valor -> pixel
const y = (v) => {
  if (vmax === vmin) return (margemTopo + (ALTURA - margemBase)) / 2;
  return margemTopo + (1 - (v - vmin) / (vmax - vmin)) * (ALTURA - margemTopo - margemBase);
};

```

Como ler essas duas funções

Ambas seguem o mesmo padrão em três passos:

1. **Normalizar:** $(t - t_0) / (t_1 - t_0)$ transforma qualquer data em um número entre 0 e 1, onde 0 é a primeira medição e 1 é a última.
2. **Escalar:** multiplicar pela largura útil converte a fração em pixels.
3. **Deslocar:** somar a margem afasta da borda, deixando espaço para os rótulos.

A diferença crucial está no $1 -$ da função `y`. Em matemática, o eixo Y cresce para cima; em tela, cresce para **baixo** — a coordenada 0 é o topo. O $1 -$ fração inverte o eixo, para que um peso maior apareça mais alto no gráfico.

DUAS DECISÕES DE PROJETO NO GRÁFICO

A folga de 15%. Sem ela, o maior valor tocaria o topo e o menor tocaria a base, e os pontos ficariam colados na borda. Ampliar a faixa em 15% para cada lado dá respiro visual. O caso `: 1` cobre a situação em que todos os valores são iguais — sem folga, a divisão $(v - \text{vmin}) / (\text{vmax} - \text{vmin})$ seria divisão por zero.

O caso de uma medição só. Com um único ponto não existe evolução para traçar. Em vez de deixar o gráfico vazio, o código desenha uma linha horizontal tracejada no nível atual e mostra a dica "Registre em outro dia para ver a linha de evolução". Preserva a leitura de gráfico sem inventar uma tendência que os dados não sustentam.

O elemento `viewBox="0 0 300 132"` merece nota: ele define um sistema de coordenadas interno independente do tamanho real na tela. O código calcula tudo em uma grade de 300 por 132, e o navegador escala o desenho para o espaço disponível. Como SVG é vetorial, não perde qualidade em nenhum tamanho de tela.

6. Dados no navegador e versionamento

6.1 O que é o localStorage

O `localStorage` é um armazenamento de pares chave/valor dentro do navegador. Diferente do cookie, ele não é enviado ao servidor a cada requisição — fica só na máquina do usuário. Guarda apenas texto, então objetos precisam ser convertidos com `JSON.stringify` ao gravar e `JSON.parse` ao ler. O limite típico é de 5 a 10 MB por site.

```
CHAVE_STORAGE = `nutribalance_data_${sessao.usuario.id}`;
```

A chave inclui o id do usuário. Isso isola os dados de contas diferentes no mesmo navegador — sem isso, dois irmãos usando o mesmo computador veriam as refeições um do outro.

6.2 Por que refeições no navegador e perfil no banco

Dado	Onde fica	Consequência
Conta, perfil, metas	Banco (MySQL)	Acompanha o usuário em qualquer dispositivo
Histórico de peso e cintura	Banco (MySQL)	Acompanha o usuário em qualquer dispositivo
Refeições do dia	<code>localStorage</code>	Ficam presas àquele navegador

ESTA É A MAIOR DÍVIDA TÉCNICA DO PROJETO — ASSUMA ANTES QUE PERGUNTEM

As refeições deveriam estar no banco, como todo o resto. Estão no `localStorage` por motivo histórico: o dashboard foi construído primeiro, sem login, e naquele momento não havia banco. Quando o login foi implementado, o perfil migrou para o servidor, mas as refeições não.

As consequências são concretas e você deve saber enunciá-las: o usuário perde as refeições ao trocar de navegador ou de computador; limpar os dados do navegador apaga o histórico alimentar; e não é possível acessar do celular o que foi registrado no computador. Reconhecer isso com clareza — e saber exatamente como resolveria — demonstra mais domínio do que tentar justificar a escolha.

6.3 Versionamento de formato e a função migrar()

Este é um dos pontos mais instrutivos do projeto. Quando o formato dos dados salvos muda, os dados que já estão no navegador do usuário continuam no formato antigo. Se o código novo tentar lê-los diretamente, ou quebra ou perde tudo.

A solução é **versionar o formato** e escrever uma função de migração. O NutriBalance já passou por três versões:

Versão	Formato
v1	Refeição "plana": os macros ficavam no próprio objeto da refeição, sem lista de itens.
v2	Refeição com lista de itens, mas com as chaves em inglês (<code>profile</code> , <code>meals</code> , <code>items</code> , <code>p100</code>).
v3	Mesma estrutura, chaves em português (<code>perfil</code> , <code>refeicoes</code> , <code>itens</code> , <code>proteina100</code>).

```
const VERSAO_DADOS = 3;

function migrar(dados, perfilDaSessao) {
  if (dados && dados.versao === VERSAO_DADOS) return dados; // ja atual, nada a fazer
  try {
    // --- v1 -> v2: cada refeicao plana vira uma refeicao com um item ---
    let v2 = dados;
    if (!dados || dados.version !== 2) {
      v2 = { version: 2, profile: {...}, meals: (dados?.meals ?? []).map(...) };
    }

    // --- v2 -> v3: traduz as chaves para portugues ---
    return {
      versao: VERSAO_DADOS,
      perfil: { nome: perfilV2.name ?? "Meu Perfil", metaKcal, ...macros },
      refeicoes: (v2?.meals ?? []).map((m) => ({
        id: m.id ?? uid(),
        tipo: m.tipo ?? "Refeição",
        data: m.data ?? isoHoje(),
        itens: (m.items ?? []).map((it) => ({
          nome: it.nome ?? "Alimento",
          gramas: Number(it.gramas) || 0,
          kcal100: Number(it.kcal100) || 0,
          proteina100: Number(it.p100) || 0, // p100 -> proteina100
          carbo100: Number(it.c100) || 0, // c100 -> carbo100
          gordura100: Number(it.g100) || 0, // g100 -> gordura100
        })),
      })),
    });
  } catch {
    return dadosVazios(perfilDaSessao); // dado irrecuperavel: recomeca limpo
  }
}
```

Três técnicas defensivas aparecem aqui e valem ser nomeadas:

- **Migração encadeada.** Um dado em v1 passa por v1→v2 e depois por v2→v3. Não é preciso escrever uma conversão direta para cada par de versões.
- **Valores padrão com `??`.** O operador de coalescência nula fornece um substituto quando o campo está ausente. Se um dado antigo não tiver `tipo`, vira "Refeição" em vez de `undefined`.
- **`try/catch` abrangente.** Se o dado estiver corrompido a ponto de a conversão falhar, o sistema recomeça vazio em vez de travar numa tela em branco.

ESTE CÓDIGO FOI TESTADO, NÃO APENAS ESCRITO

A migração v2→v3 foi verificada em execução: um `localStorage` no formato v2 antigo foi criado com uma refeição de dois itens, o dashboard foi carregado e as refeições apareceram intactas, com os macros corretos. A leitura posterior do `localStorage` confirmou versão: 3 e todas as chaves em português. Poder afirmar isso com segurança vale mais do que supor que funciona.

6.4 Quem manda quando os dois lados discordam

Se o perfil está no banco e uma cópia está no `localStorage`, o que acontece quando divergem? A regra está em `sincronizarComSessao()`:

```
function sincronizarComSessao(dados, perfilDaSessao) {
  dados.perfil.nome = perfilDaSessao.nome;           // nome: banco sempre vence
  dados.perfil.foto = perfilDaSessao.foto ?? null;    // foto: banco sempre vence
  const metasDaSessao = metasDe(perfilDaSessao);
  if (!metasIguais(dados.ultimasMetasSessao, metasDaSessao)) {
    Object.assign(dados.perfil, metasDaSessao);       // metas mudaram: adota as novas
    dados.ultimasMetasSessao = metasDaSessao;
  }
  return dados;
}
```

Nome e foto são sempre do banco, sem discussão. As metas seguem regra mais sutil: o sistema guarda um *retrato* das metas da última sincronização em `ultimasMetasSessao`. Se as metas que vêm do servidor forem diferentes desse retrato, significa que o usuário refez a personalização — e as novas prevalecem. Se forem iguais, o que está no `localStorage` é preservado, porque pode conter um ajuste manual feito no modal de perfil.

O PROBLEMA QUE ISSO RESOLVE

Sem o retrato, haveria duas opções ruins. Sempre adotar o banco apagaria o ajuste manual do usuário a cada carregamento. Nunca adotar o banco faria a personalização refeita não surtir efeito no dashboard. Guardar o valor da última sincronização permite distinguir "o servidor mudou" de "o usuário ajustou localmente" — que é exatamente a informação necessária para decidir.

7. Convenção de código em português

Todo o código do projeto — variáveis, funções, constantes, nomes de arquivo e pasta, ids de HTML, classes de CSS e colunas do banco — é escrito em português.

Camada	Estilo	Exemplo
Variáveis e funções JS/PHP	<code>camelCase</code>	<code>carregarDados</code> , <code>totaisDoDia</code>
Constantes	<code>MAIUSCULA_SNAKE</code>	<code>FATOR_ATIVIDADE</code> , <code>VERSAO_DADOS</code>
Colunas e tabelas do banco	<code>snake_case</code>	<code>meta_kcal</code> , <code>cintura_cm</code>
ids de HTML e classes de CSS	<code>kebab-case</code>	<code>lista-refeicoes</code> , <code>.grafico-card</code>

Regra: sem acento e sem cedilha em identificadores

Escreve-se `refeicao` , não `refeição` . O JavaScript até aceita acento em nome de variável, mas isso quebra em seletor de CSS, chave de `localStorage` e coluna de SQL, e gera problemas de codificação entre PHP, MySQL e navegador. Acento continua normal em **texto exibido na tela e em comentários**.

O que permanece em inglês

Jargão consolidado, onde traduzir atrapalharia mais do que ajudaria: `id` , `kcal` , `iso` , `svg` , `json` , `html` , `hash` , `tdee` , `uid` , `modal` , `link` , `app` , além das APIs do navegador e do PHP (`getElementById` , `localStorage` , `fetch` , `password_hash`).

Glossário canônico

O maior risco de uma base traduzida é a inconsistência — usar `refeicao` em um arquivo e `comida` em outro. Por isso a tradução seguiu um glossário fixo:

Inglês	Português	Inglês	Português
meal	<code>refeicao</code>	goal	<code>meta</code>
food	<code>alimento</code>	profile	<code>perfil</code>
chart	<code>grafico</code>	state	<code>estado</code>
progress	<code>progresso</code>	data	<code>dados</code>
add	<code>adicionar</code>	remove	<code>remover</code>
save / load	<code>salvar</code> / <code>carregar</code>	empty	<code>vazio</code>
render / draw	<code>mostrar</code> / <code>desenhar</code>	handle	<code>tratar</code>
show / hide	<code>mostrar</code> / <code>esconder</code>	preview	<code>previa</code>
bmr	<code>tmb</code>	totals / sum	<code>totais</code> / <code>somar</code>

8. Mapa de arquivos e como rodar

8.1 Estrutura do projeto

```
MazyOS/projetos/NutriBalance/
├── banco/
│   ├── nutribalance_completo.sql Banco inteiro: 4 tabelas + os 597 alimentos
│   ├── migracao_redefinicoes_senha.sql So a tabela nova, para banco ja criado
│   └── importar_taco.php Regera os alimentos a partir da planilha
├── dados/
│   └── Taco-4a-Edicao.xlsx Planilha original da TACO
└── site/
    ├── index.html Dashboard
    ├── login.html Tela de entrada
    ├── cadastro.html Criacao de conta
    ├── personalizacao.html Onboarding / edicao de perfil
    ├── esqueceu-senha.html Pedido do link de redefinicao
    ├── redefinir-senha.html Cadastro da senha nova (abre pelo link do e-mail)
    ├── img/ Logos
    ├── uploads/fotos/ Fotos de perfil enviadas
    ├── css/ Uma folha por area da interface
    │   ├── base.css Variaveis, reset e componentes comuns
    │   ├── dashboard.css Cabecalho, navegacao por data, refeicoes
    │   ├── perfil.css Card de perfil e foto
    │   ├── progresso.css Historico e grafico
    │   ├── alimentos.css Modal de busca de alimento
    │   ├── modal.css Modal generico
    │   └── acesso.css Login, cadastro e personalizacao
    ├── js/ Modulos ES (import/export)
    │   ├── principal.js Entry point do dashboard: boot da sessao
    │   ├── estado.js Estado compartilhado do dashboard
    │   ├── dados.js localStorage e migracao de formato
    │   ├── utilitarios.js Datas, texto e ids
    │   ├── macros.js Calculo dos macros consumidos
    │   ├── tela.js Desenho do dashboard
    │   ├── refeicoes.js Refeicoes, navegacao por data
    │   ├── alimentos.js Busca na TACO e entrada manual
    │   ├── perfil.js Modal de perfil e foto
    │   ├── calculo.js Formulas (TMB, TDEE, macros, IMC, previsao)
    │   ├── auth.js Login, cadastro, logout, guarda de sessao
    │   ├── progresso.js Historico e grafico SVG
    │   ├── personalizacao.js Previa ao vivo do onboarding
    │   ├── senha.js Botao olho: mostra/oculta campo de senha
    │   └── travas.js Bloqueia os atalhos de zoom do navegador
    └── api/
        ├── _bootstrap.php Base comum (sessao, JSON, helpers)
        ├── conexao.php Conexao PDO
        ├── registrar.php Cadastro
        ├── login.php Autenticacao
        ├── logout.php Encerrar sessao
        ├── sessao.php Estado da sessao / perfil
        ├── salvar_perfil.php Onboarding: valida e recalcula metas
        └── buscar.php Busca de alimentos na TACO
```


— salvar_medicao.php	Registra peso/cintura (upsert)
— listar_medicoes.php	Historico para o grafico
— foto.php	Upload, troca e remocao da foto
— esqueceu_senha.php	Emito o token e envia o link
— redefinir_senha.php	Valida o token e grava a senha nova
— email.php	Envio por SMTP (PHPMailer)
— email_config.php	Credenciais do SMTP (fora do Git)
— lib/PHPMailer/	Unica biblioteca externa do projeto

8.2 Como colocar para rodar

1. **Ligar o XAMPP.** No Control Panel, iniciar **Apache** e **MySQL**. Os dois precisam estar ativos.
2. **Colocar o site no Apache.** Copiar a pasta `site` para dentro do `htdocs` do XAMPP, renomeada para `nutribalance`. No ambiente de desenvolvimento usa-se um *junction* (atalho de diretório), para editar no projeto e o Apache servir ao vivo.
3. **Importar o banco.** Abrir `http://localhost/phpmyadmin`, aba Importar, escolher `nutribalance_completo.sql`. Esse arquivo já traz os 597 alimentos, as tabelas e os dados.
4. **Conferir a senha do MySQL.** No XAMPP padrão, o usuário `root` não tem senha. Se o computador tiver, ajustar `DB_PASS` em `site/api/conexao.php`.
5. **Configurar o envio de e-mail** (só para a recuperação de senha). Criar o arquivo `site/api/email_config.php` com o conteúdo indicado logo abaixo. Sem esse passo o resto do sistema roda normalmente; apenas o e-mail de redefinição não sai.
6. **Abrir** `http://localhost/nutribalance/`.

O arquivo email_config.php

Este arquivo **não está no repositório**: ele guarda uma senha, e por isso está no `.gitignore`. Em uma máquina nova é preciso criá-lo à mão, com esta estrutura:

```
<?php
const SMTP_HOST = 'smtp.gmail.com';
const SMTP_PORTA = 587;                // 587 = STARTTLS
const SMTP_USUARIO = 'suaconta@gmail.com'; // conta que ENVIA
const SMTP_SENHA = 'xxxx xxxx xxxx xxxx'; // senha de app, 16 letras
const SMTP_REMETENTE_NOME = 'NutriBalance';
const URL_SITE = 'http://localhost/nutribalance'; // sem barra no fim
```

A `SMTP_SENHA` não é a senha da conta Google. É uma **senha de app**, gerada em `myaccount.google.com/apppasswords`, e essa página só aparece se a conta tiver **verificação em duas etapas ativa**. O Gmail recusa login SMTP com a senha comum desde 2022. As 16 letras podem ser coladas com ou sem espaços — o código os remove.

O `URL_SITE` é a base do link que vai no e-mail e precisa bater com o endereço usado no navegador; se for deixado em branco, o sistema monta o endereço a partir da própria requisição.

O ERRO MAIS COMUM NA HORA DE APRESENTAR

Abrir o `index.html` com clique duplo. Não funciona: o endereço fica `file:///...` e o PHP não é executado, então login e busca de alimentos falham. Tem que ser pelo endereço `http://localhost/nutribalance/`, com Apache e MySQL ligados.

9. Perguntas prováveis da banca

As respostas abaixo estão resumidas de propósito: são o esqueleto do que dizer, não um texto para decorar. O conteúdo completo de cada uma está nas seções anteriores.

Por que o PHP não gera o HTML, como se costuma fazer?

Porque o dashboard é uma tela que muda o tempo todo sem trocar de página: adicionar um alimento move as barras de macros, trocar de dia troca a lista, registrar uma medição redesenha o gráfico. Com o PHP imprimindo a tela, cada uma dessas ações recarregaria a página inteira. Separando, o PHP responde só os dados e o JavaScript atualiza o pedaço que mudou. Há também o reaproveitamento: o `sessao.php` serve ao dashboard, à personalização e à guarda de todas as páginas — como devolve dados e não HTML, cada tela usa o que precisa. As duas arquiteturas são válidas; o que não pode é misturar as duas no mesmo projeto, e por isso os onze endpoints seguem o mesmo padrão.

Você usou bibliotecas prontas?

Uma só, o PHPMailer, para o envio de e-mail — e instalada à mão, três arquivos, sem Composer. O resto é PHP e JavaScript puros: nenhum framework, nenhum CDN, nenhuma etapa de build. O que pode parecer biblioteca são funções nativas do PHP: `password_hash`, `filter_var`, `random_bytes` e o `PDO` fazem parte da linguagem. Até o gráfico de progresso é SVG escrito na mão, sem biblioteca de gráficos.

Por que guardar o hash do token e não o token?

Porque enquanto está válido o token vale tanto quanto a senha: ele autoriza a troca. Se ficasse em texto puro e o banco vazasse, quem lesse a tabela trocaria a senha de qualquer conta com pedido pendente. Guardando o SHA-256, o vazamento não serve para nada — do hash não se volta ao token. É o mesmo raciocínio do `senha_hash`.

Por que não enviar uma senha nova pronta por e-mail?

Por dois motivos. A senha trafegaria em texto puro e ficaria guardada na caixa de entrada por tempo indeterminado. E qualquer pessoa poderia trocar a senha de outra só sabendo o e-mail dela, mesmo sem acesso à caixa — bastaria pedir a redefinição para trancar a vítima fora da conta. Com token, a senha atual continua valendo até o dono do e-mail escolher a nova.

Por que você não usou React ou outro framework?

Porque o projeto tem seis páginas e não precisa da complexidade que um framework resolve. Além disso, escrever a manipulação de DOM na mão me obrigou a entender o que o framework faria por baixo — delegação de eventos, renderização, escopo. E, na prática, não há etapa de build nem dependências: copio a pasta, ligo o XAMPP e roda em qualquer computador.

Se o cálculo já existe em JavaScript, por que repetir em PHP?

Porque o JavaScript roda no navegador do usuário, e ele pode alterá-lo. A versão em JavaScript serve para mostrar a prévia ao vivo enquanto a pessoa digita, sem ir ao servidor a cada tecla. A versão em PHP é a

que vale: quando salvo, o servidor recalcula do zero a partir dos dados brutos e ignora o valor que o navegador enviou. É a aplicação da regra de nunca confiar no cliente.

Como você protege contra SQL Injection?

Usando prepared statements em todas as consultas. O SQL vai para o banco com marcadores `?` no lugar dos valores, e os valores são enviados separadamente. Assim o banco nunca interpreta a entrada do usuário como comando — se alguém digitar `'; DROP TABLE usuarios; --`, isso vira apenas um texto a ser procurado. Também configurei `EMULATE_PREPARES => false` para garantir que a preparação é feita pelo MySQL, não simulada pelo PHP.

E se o banco de dados vazar? As senhas ficam expostas?

Não, porque as senhas não são armazenadas. Guardo apenas o hash gerado por `password_hash()`, que é um caminho só de ida — do hash não se recupera a senha. Na verificação, aplico a mesma função à senha digitada e comparo. O `password_hash` ainda aplica um salt aleatório, então dois usuários com a mesma senha têm hashes diferentes.

Como o sistema sabe que continuo logado ao mudar de página?

Por sessão. No login, o PHP cria um arquivo no servidor com o meu id e envia ao navegador apenas um identificador aleatório, dentro de um cookie. A cada requisição o navegador reenvia esse cookie e o PHP recupera quem eu sou. Os dados ficam no servidor; o navegador carrega só a chave. No login eu regenero o identificador, para evitar ataque de fixação de sessão.

De onde vem a fórmula do cálculo de calorias?

Da equação de Mifflin-St Jeor, de 1990, que calcula a taxa metabólica basal a partir de peso, altura, idade e sexo. Multiplico pelo fator de atividade para obter o gasto diário total, e aplico um ajuste de 15% para baixo ou para cima conforme o objetivo. Escolhi Mifflin-St Jeor em vez de Harris-Benedict porque é mais recente e apresenta melhor precisão para pessoas com sobrepeso, que são boa parte do público de um app de contagem de calorias.

Por que déficit de 15% e não mais?

Porque déficits agressivos aumentam a perda de massa magra, são difíceis de sustentar e podem gerar deficiência de nutrientes. A faixa geralmente recomendada é de 10% a 20%, e 15% é o meio dela. Usei percentual em vez de valor fixo porque 500 kcal a menos pesa muito mais para quem gasta 1800 do que para quem gasta 3500 — o percentual se ajusta ao tamanho da pessoa.

Por que acompanhar a cintura, se já existe o peso?

Porque o peso sozinho esconde mudanças de composição corporal. É possível manter o peso e reduzir a cintura, o que indica troca de gordura por músculo. O resumo da seção Progresso identifica esse caso e o descreve de forma neutra, sem prometer resultado.

O que acontece se dois usuários usarem o mesmo computador?

Os dados do servidor são separados pela sessão. No `localStorage`, a chave inclui o id do usuário — `nutribalance_data_3`, `nutribalance_data_9` — então as refeições de um não aparecem para o outro.

Qual a maior limitação do seu sistema?

As refeições ficam no `localStorage` e não no banco. Isso significa que trocar de navegador ou de computador faz perder o histórico alimentar. É consequência da ordem em que desenvolvi: o dashboard veio antes do login. Sei exatamente como resolver — criar as tabelas `refeicoes` e `itens_refeicao` com chave estrangeira para `usuarios`, e migrar o que estiver no navegador na primeira vez que a pessoa entrar.

Como você garantiu que o sistema funciona?

Testei os endpoints diretamente, verificando o fluxo completo de cadastro, login, salvar perfil, registrar medição, listar e sair. Testei a migração dos dados salvos criando um `localStorage` no formato antigo e conferindo que as refeições sobreviveram. E testei o banco exportando o dump e reimportando em um banco separado, para confirmar que restaura 597 alimentos e todos os usuários e medições.

10. Limitações honestas e próximos passos

Reconhecer os limites do próprio trabalho é sinal de domínio, não de fraqueza. Estas são as limitações reais do NutriBalance hoje.

Limitação	Situação e como resolver
Refeições no navegador	Não sincronizam entre dispositivos. Solução: tabelas <code>refeicoes</code> e <code>itens_refeicao</code> no banco, com migração do que existe no <code>localStorage</code> .
Fórmula duplicada	O cálculo existe em <code>calculo.js</code> e em <code>salvar_perfil.php</code> . É deliberado, mas se a fórmula mudar é preciso alterar nos dois. Há comentário nos dois arquivos alertando sobre isso.
Previsão linear	Não considera que o TDEE cai conforme o peso diminui, então tende a ser otimista em prazos longos. Solução: recalcular o TDEE em intervalos ao longo da simulação.
Sem edição de medição	Registrar de novo no mesmo dia substitui o valor, mas não há interface para apagar uma medição de outra data.
Porções caseiras aproximadas	A TACO não traz peso de porção; os valores de "1 fatia", "1 concha" seguem tabelas de medidas caseiras e servem como atalho. O usuário sempre pode digitar os gramas.
Envio de e-mail depende de SMTP externo	A recuperação de senha existe e funciona, mas é o único ponto do sistema que depende de um serviço de fora: precisa de uma conta SMTP configurada em <code>email_config.php</code> . Sem ela, o pedido é registrado no banco mas o e-mail não sai. Em rede que bloqueie a porta 587, o envio falha.
HTTP sem TLS	Em ambiente local não é problema. Em produção, seria obrigatório HTTPS e cookie de sessão com as marcações <code>Secure</code> e <code>HttpOnly</code> .

11. Glossário

Termo	Significado
API	Conjunto de endereços que um sistema oferece para outro consumir. Aqui, a pasta <code>api/</code> .
Back-end	A parte que roda no servidor: PHP e banco de dados.
Closure	Função que mantém acesso às variáveis do escopo onde foi criada, mesmo depois que ele terminou.
Cookie	Pequeno texto que o servidor pede ao navegador para guardar e reenviar a cada requisição.
Debounce	Técnica de adiar uma ação até que os disparos parem, para evitar execuções repetidas.
Delegação de eventos	Registrar um ouvinte no elemento pai em vez de um em cada filho.
DOM	<i>Document Object Model</i> : a representação da página em memória, que o JavaScript manipula.
Endpoint	Cada endereço do servidor que aceita requisições.
Front-end	A parte que roda no navegador: HTML, CSS e JavaScript.
Hash	Resultado de uma função só de ida, usada para guardar senhas sem guardar a senha.
HTTP	Protocolo de comunicação entre navegador e servidor. Não guarda estado entre requisições.
IIFE	Função criada e executada imediatamente, usada para criar escopo privado.
IMC	Índice de Massa Corporal: peso dividido pela altura ao quadrado.
JSON	Formato de texto para troca de dados estruturados entre sistemas.
localStorage	Armazenamento de texto no navegador, que persiste entre visitas e não vai ao servidor.
Macronutriente	Carboidrato, proteína ou gordura — os nutrientes que fornecem energia.
Migration	Arquivo que descreve uma alteração no formato do banco, aplicada em ordem.
PDO	<i>PHP Data Objects</i> : a interface do PHP para acessar bancos de dados.
Prepared statement	Consulta enviada ao banco com marcadores, e valores enviados à parte. Defesa contra SQL Injection.
Salt	Valor aleatório somado à senha antes do hash, para que senhas iguais gerem hashes diferentes.
Sessão	Mecanismo pelo qual o servidor reconhece o usuário entre requisições.
SQL Injection	Ataque em que a entrada do usuário é interpretada como comando SQL.

SVG	Formato de imagem vetorial descrito em texto, que não perde qualidade ao ser ampliado.
TACO	Tabela Brasileira de Composição de Alimentos, publicada pela UNICAMP.
TDEE	Gasto energético diário total: TMB multiplicada pelo fator de atividade.
Template string	Texto entre crases em JavaScript, que permite inserir valores com <code>`\${}`</code> .
TMB	Taxa metabólica basal: energia gasta em repouso absoluto.
Upsert	Operação que insere se não existe e atualiza se existe.
XSS	<i>Cross-Site Scripting</i> : ataque em que código do atacante é executado no navegador da vítima.